Tech Challenge — Fase 04 · Blog POSTECH Mobile

Aplicativo **React Native (Expo)** para a aplicação de blogging dinâmico desenvolvida ao longo do curso, somado à evolução da API Node.js criada na Fase 02 (autenticação JWT, cadastro de professores e de alunos).

Camada	Stack
Mobile	React Native 0.88 · Expo SDK 58 · React Navigation 7 · Axios · Context API
Backend	Node.js · Express 4 · Mongoose 8 · JWT · bcryptjs
Banco	MongoDB 6
Testes	Jest · Supertest · mongodb-memory-server

Sumário

1. Estrutura do repositório
2. Setup inicial
3. Rodando o backend
4. Rodando o app mobile
5. Credenciais de acesso
6. Arquitetura da aplicação
7. Guia de uso
8. API REST
9. Testes
10. Solução de problemas

Estrutura do repositório

```
tech-challenge-fase4/
├── backend/                API REST (evolução da Fase 02)
│   └── src/
│       ├── app.js              Express, CORS, registro das rotas
│       ├── server.js           Entrada de produção (conecta no Mongo real)
│       ├── devServerMemoria.js  Entrada de desenvolvimento (Mongo em memória)
│       ├── config/             Conexão com o MongoDB
│       ├── models/             Post e User (professor | aluno)
│       ├── controllers/        Regras de posts, usuários e autenticação
│       ├── middlewares/        autenticar / autorizar + tratamento de erro
│       ├── routes/             /auth, /posts, /professores, /alunos
│       ├── seeds/              Popular o banco com dados de demonstração
│       ├── utils/              Paginação
│       └── tests/              Suíte Jest + Supertest
├── mobile/                 Aplicativo Expo
│   ├── App.js                  Providers (SafeArea + Auth) e navegação
│   └── src/
│       ├── api/                Cliente Axios e services REST
│       ├── components/         Botão, input, card, busca, paginador, feedbacks
│       ├── contexts/           AuthContext (sessão, login, logout)
│       ├── hooks/              useListaPaginada, useRecarregarAoFocar
│       ├── navigation/         RootNavigator (tabs + stacks)
│       ├── screens/            Telas de blog, admin, login e perfil
│       ├── theme/              Tokens de cor, espaçamento e tipografia
│       └── utils/              Formatadores e validações
├── docs/                   Documentação complementar da entrega
└── docker-compose.yml      MongoDB + API
```

Setup inicial

> Máquina nova, ou algum passo falhando? O **SETUP.md** tem o guia completo, com verificação de versões, instalação por sistema operacional e solução de problemas.

Pré-requisitos: **Node.js 22.13+** (exigência do Expo SDK 58 / React Native 0.88 — veja `engines`), **npm** e um dos caminhos de banco abaixo.

```
git clone <url-do-repositorio>
cd tech-challenge-fase4

# o .env não vem no clone
cp backend/.env.example backend/.env

# dependências
cd backend && npm ci && cd ..
cd mobile  && npm ci && cd ..
```

Rodando o backend

Opção A — Docker (recomendado para a entrega)

```
docker compose up -d --build
```

Sobe MongoDB (`27017`) e a API (`http://localhost:3001`). Na primeira execução, popule o banco:

```
docker compose exec api npm run seed
```

Opção B — MongoDB em memória (sem instalar nada)

```
cd backend
npm run dev:memory
```

A API sobe em `http://localhost:3001` já populada com professores, alunos e posts de exemplo. Os dados vivem apenas enquanto o processo estiver aberto — ideal para desenvolver e gravar a demonstração.

Opção C — MongoDB local instalado

```
cd backend
cp .env.example .env     # ajuste MONGODB_URI para mongodb://localhost:27017/blogdb
npm run seed
npm run dev
```

Variáveis de ambiente (`backend/.env`)

Variável	Para que serve
`PORT`	Porta do Express dentro do container (padrão `3000`)
`MONGODB_URI`	String de conexão do MongoDB
`JWT_SECRET`	Segredo usado para assinar os tokens
`JWT_EXPIRES_IN`	Validade do token (padrão `7d`)
`SEED_PROFESSOR_EMAIL` / `SEED_PROFESSOR_SENHA`	Credenciais do professor criado pelo seed

Rodando o app mobile

```
cd mobile
npm start          # abre o Metro com o QR code
npm run android    # emulador ou dispositivo Android conectado
npm run web        # navegador (útil para verificação rápida)
```

Para onde o app aponta

O app resolve a URL da API sozinho:

Ambiente	URL usada
Emulador Android	`http://10.0.2.2:3001`
iOS / Web	`http://localhost:3001`
Celular físico (Expo Go)	defina `EXPO_PUBLIC_API_URL`

Para celular físico, copie `mobile/.env.example` para `mobile/.env` e aponte para o IP da sua máquina na rede:

```
EXPO_PUBLIC_API_URL=http://192.168.0.10:3001
```

> `10.0.2.2` é o apelido que o emulador do Android usa para o `localhost` da máquina hospedeira. `localhost` dentro do emulador seria o próprio emulador.

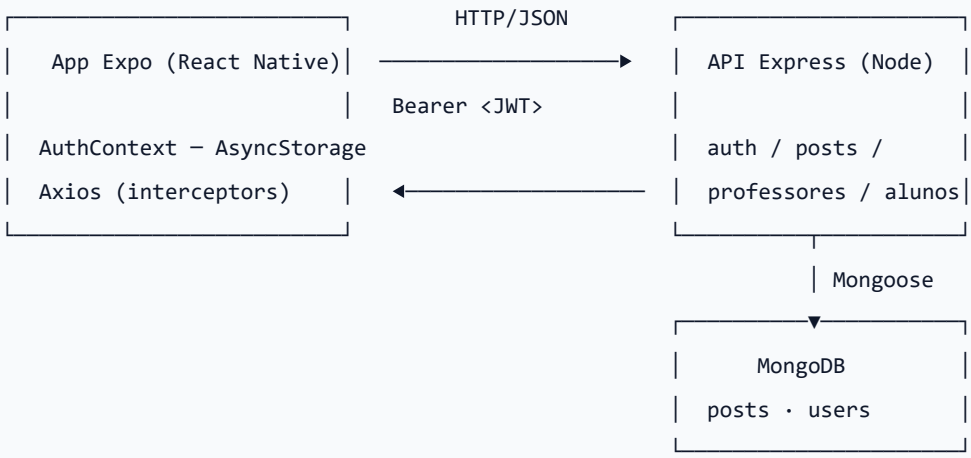
Credenciais de acesso

Criadas pelo seed (e pelo modo em memória):

Perfil	E-mail	Senha
Professor	`professor@postech.com`	`postech2026`
Professor	`ana.ribeiro@postech.com`	`postech2026`
Aluno	`bruna.lima@aluno.postech.com`	`postech2026`

Arquitetura da aplicação

Visão geral



Backend

- **Modelo único de usuário.** `User` guarda professores e alunos separados pelo campo `role`. As rotas `/professores` e `/alunos` são geradas pela mesma fábrica (`criarControllerDeUsuario`), o que evita duplicar seis handlers quase idênticos e garante que os dois recursos se comportem igual.
- **Senhas.** Hash com bcrypt em um hook `pre('save')`; o campo tem `select: false` e é removido no `toJSON`, então nunca sai numa resposta.
- **Autenticação.** `POST /auth/login` devolve um JWT assinado com `{ id, role }`. O middleware `autenticar` valida o token e recarrega o usuário; `autorizar('professor')` barra quem não tem o papel certo.
- **Permissões.** Leitura de posts é pública (alunos e visitantes). Criar, editar e excluir post, e todo o CRUD de professores/alunos, exigem professor autenticado.
- **Paginação.** `utils/paginacao.js` normaliza `?page` / `?limit` e devolve um bloco `meta` com `totalPages`, `hasNextPage` e `hasPrevPage` — consumido direto pelo componente `Paginador` do app.
- **Erros.** `tratarErro` traduz `ValidationError`, `CastError` e chave duplicada (`11000`) em 400/409 com mensagem legível, em vez de vazar stack trace.

Mobile

- **Navegação.** `RootNavigator` monta um stack raiz com as *tabs* e a tela de Login como modal. As tabs são **Blog**, **Admin** (renderizada apenas quando `role === 'professor'`) e **Perfil**, cada uma com seu próprio stack.
- **Estado global.** `AuthContext` (Context API + hooks) guarda o usuário, persiste token e perfil no `AsyncStorage`, revalida a sessão no `GET /auth/me` ao abrir o app e derruba o login automaticamente quando a API responde 401.

- **Camada de rede.** Um único `axios` com interceptor de request (injeta o `Bearer`), interceptor de response (401 → logout) e `extrairMensagemDeErro`, que transforma qualquer falha — inclusive `Network Error` — em texto que o usuário entende.
- **Reaproveitamento de telas.** `UserListScreen` e `UserFormScreen` atendem professores e alunos; o que muda é o parâmetro `role` da rota. As rotas continuam separadas (`ProfessorList`, `AlunoForm`, …), então a navegação fica explícita sem duplicar código.
- **Listagens.** O hook `useListaPaginada` concentra paginação, busca com *debounce* de 400 ms, *pull-to-refresh*, estado de erro e descarte de respostas fora de ordem. Os três listamentos do app (posts, posts do admin, usuários) usam esse mesmo hook.
- **Design system enxuto.** `src/theme` centraliza cores, espaçamentos, raios e sombras; os componentes de UI (`AppButton`, `AppInput`, `PostCard`, `SearchBar`, `UserRow`, `Paginador`, `Feedback`) consomem esses tokens.

Guia de uso

Tela	Como chegar	O que faz
Lista de posts	Aba *Blog*	Lista paginada com título, autor, data e resumo. Campo de busca filtra por título, conteúdo ou autor. *Pull-to-refresh* disponível.
Leitura do post	Toque em um card	Conteúdo completo, autor e datas. Para professores, mostra os botões de editar e excluir.
Criar post	Botão flutuante (+) na aba *Blog* ou *Nova postagem* no painel	Formulário de título, autor e conteúdo com validação e contador de caracteres.
Editar post	*Editar* na leitura do post ou na listagem administrativa	Carrega os dados atuais e salva as alterações.
Painel administrativo	Aba *Admin* (só professores)	Atalhos para postagens, professores e alunos.
Postagens (admin)	*Admin → Postagens*	Lista com filtro e botões de editar/excluir por post, com confirmação.
Professores	*Admin → Professores*	Listagem paginada com editar e excluir.
Novo/editar professor	Botão *Novo professor* ou *Editar*	Nome, e-mail, matéria e senha. Na edição, senha em branco mantém a atual.
Alunos	*Admin → Alunos*	Mesmo fluxo dos professores, com o campo *Turma*.
Login	Botão *Entrar* na home ou na aba *Perfil*	E-mail e senha, com exibir/ocultar senha e mensagens de erro da API.
Perfil	Aba *Perfil*	Dados da conta, papel, permissões, URL da API em uso e logout.

Regras de permissão no app: visitantes e alunos leem posts; apenas professores veem o botão de criar, os botões de editar/excluir e a aba *Admin*. A restrição também é aplicada no backend — o app apenas reflete o que a API garante.

API REST

Base: `http://localhost:3001`

Autenticação

Método	Rota	Acesso	Descrição
`POST`	`/auth/login`	público	Recebe `{ email, senha }`, devolve `{ token, user }`
`GET`	`/auth/me`	autenticado	Usuário dono do token

Posts

Método	Rota	Acesso	Descrição
`GET`	`/posts?page=1&limit=10&q=termo`	público	Lista paginada
`GET`	`/posts/search?q=termo`	público	Busca por palavra-chave
`GET`	`/posts/:id`	público	Post por id
`POST`	`/posts`	professor	Cria post
`PUT`	`/posts/:id`	professor	Edita post
`DELETE`	`/posts/:id`	professor	Exclui post

Professores e alunos

Mesmo contrato nos dois recursos (`/professores` e `/alunos`), todos restritos a professor autenticado:

Método	Rota	Descrição
`GET`	`/professores?page=1&limit=10&q=nome`	Listagem paginada
`GET`	`/professores/:id`	Registro por id
`POST`	`/professores`	Cadastra (`nome` , `email` , `senha` , `materia` / `turma`)
`PUT`	`/professores/:id`	Edita (senha opcional)
`DELETE`	`/professores/:id`	Exclui

Formato das respostas

```
// sucesso em listagem
{
  "success": true,
  "count": 10,
  "meta": { "page": 1, "limit": 10, "total": 34, "totalPages": 4, "hasNextPage": true, "hasPrevPage":
false },
  "data": [ /* ... */ ]
}

// erro
{ "success": false, "message": "E-mail ou senha inválidos" }
```

Exemplo rápido

```
TOKEN=$(curl -s -X POST http://localhost:3001/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"professor@postech.com","senha":"postech2026"}' | jq -r .data.token)

curl -X POST http://localhost:3001/posts \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"titulo":"Aula de hoje","conteudo":"Resumo da aula."}'
```

Testes

```
cd backend
npm test
```

22 testes cobrindo login, expiração/invalidez de token, leitura pública de posts, bloqueio de escrita para aluno e visitante, CRUD de professores e alunos, paginação, e-mail duplicado e proteção contra o professor excluir a si mesmo. A suíte usa `mongodb-memory-server`, então não precisa de um MongoDB rodando.

Solução de problemas

Sintoma	Causa provável	Solução
`Network Error` no app	API fora do ar ou URL errada	Confirme `http://localhost:3001` no navegador; no emulador Android a URL correta é `10.0.2.2`
App conecta no web mas não no emulador	Emulador não enxerga `localhost`	Use `10.0.2.2` (padrão do app) ou defina `EXPO_PUBLIC_API_URL`
Celular físico não conecta	Expo Go está fora da máquina	Defina `EXPO_PUBLIC_API_URL` com o IP da máquina na rede e libere a porta 3001 no firewall
`401` logo após logar	`JWT_SECRET` mudou entre reinícios	Fixe `JWT_SECRET` no `.env` e faça login de novo
Login falha com o seed	Banco vazio	Rode `npm run seed` (ou use `npm run dev:memory`)
Metro travado após instalar pacote	Cache do bundler	`npx expo start -c`
Erro de engine ou sintaxe no Metro	Node abaixo de 22.13	Atualize o Node — o Expo SDK 58 exige `^22.13.0 || ^24.3.0 || >=26`
Alterei `mobile/.env` e nada mudou	`EXPO_PUBLIC_*` entra no bundle	`npx expo start -c`

Entrega

- **Código-fonte:** este repositório.
- **Documentação:** este README + `docs/`.
- **Apresentação gravada:** vídeo de até 15 min demonstrando o app e os detalhes técnicos.
